ADVANCED SQL QUERY SYSTEM

Project Title: Snapdeal – E-Commerce Order Management System

Objectives

1. Implement subqueries and nested queries.
2. Find products above average price.
3. Identify customers with maximum purchases.
4. Generate complex business queries.
5. Prepare an advanced SQL report**.**

1: Implement Subqueries and Nested Queries

```
SQL> SELECT *
  2  FROM PRODUCT
  3  WHERE PRICE > (
  4      SELECT AVG(PRICE)
  5      FROM PRODUCT
  6  );

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_ID      PRICE      STOCK
----------- ---------- ----------
DESCRIPTION
--------------------------------------------------------------------------------
         1
Laptop
          1      50000         10
Laptop for daily use


PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
CATEGORY_ID      PRICE      STOCK
----------- ---------- ----------
DESCRIPTION
--------------------------------------------------------------------------------
         2
```

2: Find Products Above the Average Price

```
SQL> SELECT
  2      PRODUCT_ID,
  3      PRODUCT_NAME,
  4      PRICE
  5  FROM
  6      PRODUCT
  7  WHERE
  8      PRICE > (
  9          SELECT AVG(PRICE)
 10          FROM PRODUCT
 11      )
 12  ORDER BY
 13      PRICE DESC;

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
         1
Laptop
     50000

         2
Mobile
     20000

PRODUCT_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
     PRICE
----------
```

3: Identify Customers with Maximum Purchases

```
SQL> SELECT C.CUSTOMER_ID, C.CUSTOMER_NAME, SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE
  2  FROM CUSTOMER C, ORDERS O
  3  WHERE C.CUSTOMER_ID = O.CUSTOMER_ID
  4  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
  5  HAVING SUM(O.TOTAL_AMOUNT) = (
  6      SELECT MAX(TOTAL_PURCHASE)
  7      FROM (
  8          SELECT SUM(TOTAL_AMOUNT) AS TOTAL_PURCHASE
  9          FROM ORDERS
 10          GROUP BY CUSTOMER_ID
 11      )
 12  );

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_PURCHASE
--------------
        101
Rahul
         50000
```

4: Generate Complex Business Queries

Customer Order and Payment Details

```
SQL> SELECT
  2      C.CUSTOMER_NAME,
  3      O.ORDER_ID,
  4      P.PRODUCT_NAME,
  5      O.QUANTITY,
  6      O.TOTAL_AMOUNT,
  7      O.ORDER_STATUS,
  8      PY.PAYMENT_METHOD,
  9      PY.PAYMENT_STATUS
 10  FROM ORDERS O
 11  INNER JOIN CUSTOMER C ON O.CUSTOMER_ID = C.CUSTOMER_ID
 12  INNER JOIN PRODUCT P ON O.PRODUCT_ID = P.PRODUCT_ID
 13  INNER JOIN PAYMENT PY ON O.ORDER_ID = PY.ORDER_ID
 14  ORDER BY O.ORDER_ID ASC;

CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY TOTAL_AMOUNT ORDER_STATUS         PAYMENT_METHOD
---------- ------------ -------------------- ------------------------------
PAYMENT_STATUS
--------------------
Rahul
      1001
Laptop

CUSTOMER_NAME
--------------------------------------------------------------------------------
  ORDER_ID
----------
PRODUCT_NAME
--------------------------------------------------------------------------------
  QUANTITY TOTAL_AMOUNT ORDER_STATUS         PAYMENT_METHOD
---------- ------------ -------------------- ------------------------------
PAYMENT_STATUS
--------------------
         1        50000 Delivered            UPI
Paid
```

5: Prepare Advanced SQL Reports

1. Customer-wise Purchase Report

```
SQL> SELECT
  2      C.CUSTOMER_ID,
  3      C.CUSTOMER_NAME,
  4      COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
  5      SUM(O.TOTAL_AMOUNT) AS TOTAL_PURCHASE,
  6      AVG(O.TOTAL_AMOUNT) AS AVERAGE_PURCHASE
  7  FROM CUSTOMER C
  8  JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID
  9  GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 10  ORDER BY TOTAL_PURCHASE DESC;

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_PURCHASE
------------ -------------- ----------------
       101
Rahul
           1          50000            50000

       102
Priya
           1          20000            20000

CUSTOMER_ID
-----------
CUSTOMER_NAME
--------------------------------------------------------------------------------
TOTAL_ORDERS TOTAL_PURCHASE AVERAGE_PURCHASE
------------ -------------- ----------------

       103
Arun
           1           3000             3000

       104
```

2. Order Status Report

```
SQL> SELECT
  2      ORDER_STATUS,
  3      COUNT(*) AS TOTAL_ORDERS,
  4      SUM(TOTAL_AMOUNT) AS TOTAL_AMOUNT
  5  FROM
  6      ORDERS
  7  GROUP BY
  8      ORDER_STATUS
  9  ORDER BY
 10      TOTAL_AMOUNT DESC;

ORDER_STATUS         TOTAL_ORDERS TOTAL_AMOUNT
-------------------- ------------ ------------
Delivered                       2        51000
Shipped                         1        20000
Pending                         1         3000
```

3. Payment Status Report

```
SQL> SELECT
  2      PAYMENT_STATUS,
  3      COUNT(*) AS TOTAL_PAYMENTS,
  4      SUM(AMOUNT) AS TOTAL_AMOUNT
  5  FROM
  6      PAYMENT
  7  GROUP BY
  8      PAYMENT_STATUS
  9  ORDER BY
 10      TOTAL_AMOUNT DESC;

PAYMENT_STATUS       TOTAL_PAYMENTS TOTAL_AMOUNT
-------------------- -------------- ------------
Paid                              3        71000
Pending                           1         3000
```

Conclusion:

Thus, the Advanced SQL Query System was successfully implemented using Oracle SQL. Subqueries and nested queries were used to retrieve important information. Products above average price and customers with maximum purchases were identified. Complex business queries and advanced SQL reports were generated successfully. This system helps in analyzing business data and making better business decisions.